

Deep Generative Models

Complete Exam Preparation Answer Bank

Exam Preparation Notes

Covers: mathematical foundations, VAEs, GANs, GAN variants, VAE variants, derivations, and numericals

Format: All questions present in the uploaded source are answered below. Derivations are shown step by step, numerical answers include working, and standard equations are written explicitly where the source question refers to an equation from the lecture notes.

Source basis: the uploaded Sample Questions and Homework PDF, whose sections cover Introduction and Mathematical Foundations, Deep Generative Architectures, MLE and Latent Variable Models, GANs, GAN variants, and VAE variants.

Contents

- **Part I** — Introduction and Mathematical Foundations (Q1–Q8)
- **Part II** — Deep Generative Architectures (Q1–Q6)
- **Part III** — MLE and Latent Variable Models (Q1–Q4)
- **Part IV** — Generative Adversarial Networks (Q1–Q9)
- **Part V** — Variants of GAN (Q1–Q8)
- **Part VI** — Variants of VAE (Q1–Q6)
- **Final Rapid Revision** — formulas, numerical answers, and exam checklist

Notation used throughout

- x = observed data y = label/class z = latent variable
- p = true/data distribution p_θ = model distribution q_ϕ = variational approximation
- D = discriminator G = generator E = encoder
- $\log$ denotes the natural logarithm unless stated otherwise

Contents

PART I - INTRODUCTION AND MATHEMATICAL FOUNDATIONS	5
Q1. Discuss the major goals of AI - perception, reasoning, planning, communication, creativity, learning, and control/manipulation - and explain how Generative AI is contributing to these goals.	6
Q2. Explain the major capabilities of Generative AI with its major applications. Explain how the scaling of Generative AI models depends on data, model parameters, computational resources, memory, and distributed training. Discuss why scaling becomes increasingly challenging as model size increases.	7
Q3. Differentiate between discriminative and generative models in terms of the probability distributions they learn, their objectives, and their applications. Explain how a generative model can also perform a discriminative task using the relationship between joint and conditional probability distributions.	8
Q4. Explain the major challenges in deep generative modeling, particularly for high-dimensional data such as images. Explain how these challenges affect the ability to generate realistic samples and perform discriminative tasks.	9
Q5. Explain the fundamental rules of probability, including joint probability, conditional probability, the sum rule, and the product rule. Derive Bayes theorem from the product rule and explain the roles of prior, likelihood, posterior, and normalization constant.	10
Q6. Explain the Gaussian distribution and its importance in probability theory. For a dataset of i.i.d. observations, explain how Maximum Likelihood Estimation (MLE) can be used to estimate the mean and variance of a Gaussian distribution.	11
Q7. Explain the fundamental concepts of information theory. Define self-information, Shannon entropy, KL divergence, and cross-entropy. Discuss the properties of KL divergence and explain the relationship between cross-entropy and KL divergence.	12
Q8. Medical diagnosis: Cancer C_1 has prior 0.10 and likelihood 0.80; No Cancer C_2 has prior 0.90 and likelihood 0.20. Using Bayes theorem, calculate posterior probabilities and classify the X-ray.	13
PART II - DEEP GENERATIVE ARCHITECTURES	14
Q1. Differentiate between discriminative $p_\theta(Y X)$, generative $p_\theta(X)$, and conditioned generative $p_\theta(X Y)$. Explain their objectives with examples. Also solve the classifier probability question.	15
Q2. Discuss the importance of sampling, interpolation, and disentanglement in exploring the learned manifold of a deep generative model. Illustrate each concept with an example.	16
Q3. Compare GANs, VAEs, and Denoising Diffusion Models in terms of architecture, training mechanism, sampling process, and ability to generate new data.	17
Q4. Explain the architecture and working of a Variational Autoencoder (VAE). Discuss how the encoder predicts the mean and covariance of a multivariate normal distribution, how the decoder reconstructs the input, and why the latent distribution is encouraged to follow $\mathcal{N}(0, I)$	18
Q5. Explain why direct sampling from $\mathcal{N}(\mu(X), \Sigma(X))$ creates a problem during backpropagation in a VAE. Explain the reparameterization trick and show how it enables gradient-based training.	19

Q6. One-dimensional latent variable: $\mu=2$, $\sigma=0.5$, $\epsilon=-1.2$, with $z=\mu+\sigma*\epsilon$. Decoder: $x'=3z+1$. Calculate z and x'	20
PART III - MLE AND LATENT VARIABLE MODELS	21
Q1. Explain the motivation for using latent variable models. For the image example, answer (a)-(d).	22
Q2. For a latent variable model, explain why Jensen inequality applies, introduce $q(z)$, derive the ELBO, and explain why it is a lower bound.	23
Q3. For the relationship $\log p(x) = \text{ELBO} + D_{\text{KL}}(q(z x) p(z x))$, explain non-negativity, derive the relationship, state the equality condition, and explain what happens as $q_{\phi}(z x)$ improves.	24
Q4. A latent variable z has four possible configurations with joint probabilities $p(x, z_1)=0.10$, $p(x, z_2)=0.25$, $p(x, z_3)=0.05$, $p(x, z_4)=0.20$. Calculate the marginal, posterior probabilities, the most probable configuration, and interpret the result.	25
PART IV - GENERATIVE ADVERSARIAL NETWORKS	26
Q1. Explain the architecture and working principle of a GAN. Describe G and D , the minimax game, and the objective function.	27
Q2. Discriminator outputs $D(x)=0.9$ for a real sample and $D(G(z))=0.2$ for a fake sample. Calculate $L_D = -[\log D(x) + \log(1 - D(G(z)))]$ and interpret it.	28
Q3. For x_1, x_2 real with $D=0.9, 0.8$ and two fake samples with $D=0.2, 0.1$, calculate the discriminator objective $V(G, D)$ using natural logarithms. Determine whether D is performing well.	29
Q4. Generator outputs $D(G(z))=[0.1, 0.3, 0.7, 0.5]$. Using $L_G = -(1/4) \sum \log D(G(z_i))$, calculate the generator loss, interpret it, then recalculate for $[0.6, 0.7, 0.8, 0.9]$	30
Q5. Discuss major practical GAN training challenges, including unstable optimization and mode collapse. Explain oscillating losses and empirical mitigation approaches.	31
Q6. Consider a GAN with an optimal discriminator D^* . Show that the value function can be expressed using Jensen-Shannon Divergence (JSD). Define JSD, state the optimum p_G , and the corresponding objective value.	32
Q7. Explain why evaluating generative models is challenging and how evaluation depends on the intended task. Cover density estimation, sampling, latent representation learning, and downstream applications.	33
Q8. Explain the Inception Score (IS). Discuss high sharpness and high diversity using $c(y x)$ and $c(y)$	34
Q9. Explain FID, write the formula, explain its terms, interpret a low FID, and compare it conceptually with IS.	35
PART V - VARIANTS OF GAN	36
Q1. Explain the architecture and working principle of a Conditional GAN (cGAN). How does conditioning on class information modify a conventional GAN? Explain text-to-image use.	37
Q2. Explain the motivation behind BiGAN. Describe how G and E learn the joint distributions of (X, Z) , and state the inverse-mapping condition.	38
Q3. CoGAN for two semantically related but unpaired domains X_A and X_B : explain learning without correspondence supervision, weight sharing, and one limitation.	39

Q4. Explain CycleGAN for unpaired image-to-image translation. Define $G_{A \rightarrow B}$ and $G_{B \rightarrow A}$, adversarial loss, cycle consistency, identity loss, and a limitation of cycle loss alone.	40
Q5. Explain linear interpolation in GAN latent space and steerability. Given z_1, z_2 and α in $[0,1]$, derive the interpolated vector and explain transformations such as zooming, shifting, brightness, and rotation.	41
Q6. Explain BiGAN motivation, compare with conventional GAN, roles of G/E/D, joint discrimination, and how a trained BiGAN supports generation and inference.	42
Q7. Explain the problem of image-to-image translation $X \leftrightarrow Y$ without paired examples. Describe CycleGAN architecture and adversarial training in both directions.	43
Q8. Explain cycle consistency and derive the complete CycleGAN objective. Explain the role of λ and both adversarial losses.	44
PART VI - VARIANTS OF VAE	45
Q1. How does a Convolutional VAE address the curse of dimensionality? Illustrate the architecture.	46
Q2. Explain the architecture and objective of a Conditional VAE (CVAE). How does conditioning influence the latent representation and generated output?	47
Q3. Explain β -VAE, write the modified objective, explain β , and discuss disentanglement.	48
Q4. Explain the motivation for Ladder VAE (LVAE). How does it learn a hierarchical latent representation? Compare conventional hierarchical VAE and ladder architecture.	49
Q5. β -VAE reconstruction loss = 18.5 and KL loss = 2.4. Objective = $L_{\text{reconstruction}} + \beta L_{\text{KL}}$. Calculate for $\beta=1,2,4$ and explain the effect of increasing β	50
Q6. A vanilla VAE uses a fully connected layer from $64 \times 64 \times 3$ to 512. A convolutional VAE first reduces the image to $8 \times 8 \times 64$ and then uses the same 512-dimensional fully connected layer. Calculate weights and the reduction.	51
FINAL RAPID REVISION	52

PART I - INTRODUCTION AND MATHEMATICAL FOUNDATIONS

The source begins with major goals of AI, scaling of Generative AI, generative versus discriminative modeling, high-dimensional generative modeling, probability, Gaussian MLE, information theory, and a Bayes numerical.

Q1. Discuss the major goals of AI - perception, reasoning, planning, communication, creativity, learning, and control/manipulation - and explain how Generative AI is contributing to these goals.

Answer. Artificial Intelligence aims to build systems that can perceive the world, form useful internal representations, reason about alternatives, act toward goals, communicate with humans, learn from experience, and manipulate digital or physical environments. Generative AI contributes strongly because it learns data distributions and can produce new content rather than only assigning labels.

1. **Perception:** Perception is the ability to interpret inputs such as images, audio, video, text, and sensor measurements. Generative models can learn rich representations of such inputs and can reconstruct, synthesize, denoise, complete, or transform them. For example, a diffusion model can learn the visual structure of images and then generate an image from a text description.
2. **Reasoning:** Reasoning requires forming relationships among facts and drawing conclusions. Large generative models can represent patterns from large corpora and generate intermediate explanations, hypotheses, or candidate solutions. In practical systems, generation is often combined with retrieval, tools, search, or symbolic checks so that the generated reasoning is grounded in external evidence.
3. **Planning:** Planning means selecting a sequence of actions that moves a system from an initial state to a desired goal. Generative models can propose candidate action sequences, simulate possible outcomes, or generate sub-goals. The generated plan can then be verified by a planner, simulator, or constraint checker.
4. **Communication:** Generative models naturally support language generation, translation, summarization, dialogue, question answering, and multimodal interaction. This allows a machine to communicate information in a form suitable for a human or another agent.
5. **Creativity:** Generative AI can synthesize novel text, images, music, video, 3D assets, software code, and design candidates. The key point is that the output is newly generated from learned statistical structure rather than simply retrieved as an exact stored example.
6. **Learning:** Generative models learn a representation of the underlying data distribution. Learning may be supervised, weakly supervised, or unsupervised. Latent-variable models are especially useful because they can discover hidden factors of variation and provide compact representations.
7. **Control and manipulation:** A generative model can serve as a simulator or world model, predicting plausible future states and helping an agent choose actions. In robotics, for example, a model can generate predicted observations under candidate actions. The actual control loop still requires feedback and safety constraints.

Overall, Generative AI expands AI from classification and prediction toward synthesis and simulation. It is therefore useful as a component of systems that perceive, reason, plan, communicate, create, learn, and act.

Exam Tip

In an exam, connect each AI goal to one clear generative capability and one concrete application. This makes the answer both conceptual and application-oriented.

Q2. Explain the major capabilities of Generative AI with its major applications. Explain how the scaling of Generative AI models depends on data, model parameters, computational resources, memory, and distributed training. Discuss why scaling becomes increasingly challenging as model size increases.

Answer. Generative AI learns a probability distribution over data and can then sample or condition the learned distribution to create new outputs. Its major capabilities include text generation, image generation, audio/music synthesis, video generation, code generation, multimodal generation, data augmentation, completion, inpainting, style transfer, and simulation.

- **Text:** writing, summarization, translation, tutoring, question answering, code generation.
- **Images:** text-to-image, image completion, editing, super-resolution, synthetic data generation.
- **Audio and music:** speech synthesis, voice conversion, music generation, sound effects.
- **Video:** text-to-video, video continuation, editing, avatar generation.
- **Multimodal:** generating or transforming combinations of text, images, audio, and video.
- **Scientific/engineering uses:** molecular design, synthetic datasets, simulation, inverse design, and reconstruction.

Scaling depends on several interacting resources:

- **Data scale:** Larger models generally need more diverse, high-quality data. Insufficient or repetitive data limits generalization and can increase overfitting or memorization.
- **Model parameters:** Increasing parameters increases representational capacity. More parameters allow the model to represent more complex functions, but also increase training and inference cost.
- **Compute:** Training requires matrix operations over many examples and many optimization steps. Compute therefore grows with model size, data size, sequence length, and number of training steps.
- **Memory:** Parameters, optimizer states, activations, gradients, and temporary buffers must be stored. Optimizer state can require several times the memory of the parameter tensors.
- **Distributed training:** Large models are split across multiple accelerators using data parallelism, tensor/model parallelism, or pipeline parallelism. Communication and synchronization become major system costs.

Why scaling gets harder: Memory pressure grows because parameters and training states may no longer fit on one device. Communication overhead grows as devices exchange activations, gradients, or parameters. Training becomes more sensitive to data quality, optimization stability, and numerical precision. Inference also becomes expensive because every generated token or output sample may require many large matrix operations. The practical bottleneck shifts from pure arithmetic to the complete system: memory bandwidth, network bandwidth, synchronization, storage, and reliability.

Exam Tip

Remember the central idea: scaling is not only about buying a larger model. Data, parameters, compute, memory, and communication must scale together.

Q3. Differentiate between discriminative and generative models in terms of the probability distributions they learn, their objectives, and their applications. Explain how a generative model can also perform a discriminative task using the relationship between joint and conditional probability distributions.

Aspect	Discriminative model	Generative model
Distribution learned	Typically $p_{\theta}(y x)$	Typically $p_{\theta}(x)$, or a joint model $p_{\theta}(x, y)$
Main objective	Predict the correct label for an input	Model how data are distributed and/or how x and y are jointly generated
Typical output	Class label or class probabilities	New samples, likelihoods, latent variables, reconstructions, or conditional samples
Examples	Logistic regression, standard classifier, neural classifier	VAE, GAN, diffusion model, autoregressive model
Applications	Classification, ranking, decision boundaries	Synthesis, completion, simulation, density modeling, representation learning

Answer. A discriminative model directly models the conditional distribution $p(y | x)$. A generative model instead models $p(x)$ or the joint $p(x, y)$. Because joint and conditional probabilities are related by Bayes rule, a generative model can be converted into a classifier when $p(x, y)$ and the class priors are available.

$$p(y | x) = \frac{p(x, y)}{p(x)}$$

$$p(y | x) = \frac{p(x | y)p(y)}{\sum_y p(x | y)p(y)}$$

Thus, for a fixed observation x , the class with the largest posterior probability can be chosen. A simple example is a Gaussian generative classifier such as LDA/QDA, where a class-conditional density $p(x | y)$ is modeled for every class and Bayes rule converts these densities into class probabilities.

Q4. Explain the major challenges in deep generative modeling, particularly for high-dimensional data such as images. Explain how these challenges affect the ability to generate realistic samples and perform discriminative tasks.

Answer. High-dimensional data create a very large sample space. A 64 x 64 RGB image already has 64 x 64 x 3 pixel values; the number of possible images is enormous. Learning a useful distribution over that space is therefore much harder than learning a distribution over a small set of scalar variables.

- **Curse of dimensionality:** Data become sparse in high-dimensional space, so enormous amounts of training data may be needed.
- **Complex dependencies:** Pixels are not independent; edges, shapes, textures, object parts, lighting, and semantics interact across many spatial scales.
- **Likelihood difficulty:** Exact likelihoods may be hard or expensive to compute for complex models.
- **Optimization difficulty:** Deep non-convex objectives can have unstable gradients and sensitive hyperparameters.
- **Mode coverage:** A model may generate sharp samples but miss important modes of the real distribution, or generate diverse outputs that lack realism.
- **Representation quality:** A poor latent representation can hurt interpolation, controllability, and downstream classification.

These challenges affect generation because unrealistic or repeated samples appear when the learned distribution is poor. They affect discrimination because a generative representation may fail to preserve class-relevant factors. Conversely, a well-learned generative representation can provide useful features for classification, clustering, semi-supervised learning, or retrieval.

Q5. Explain the fundamental rules of probability, including joint probability, conditional probability, the sum rule, and the product rule. Derive Bayes theorem from the product rule and explain the roles of prior, likelihood, posterior, and normalization constant.

Answer. The fundamental rules are the building blocks for probabilistic modeling.

1. **Joint probability:** $p(x, y)$ is the probability that x and y occur together.
2. **Conditional probability:** $p(x | y)$ is the probability of x given that y is known.

$$p(x | y) = \frac{p(x, y)}{p(y)}, \quad p(y) > 0$$

3. **Sum rule (marginalization):** To obtain the marginal probability of x , sum over all discrete values of y or integrate over continuous y .

$$p(x) = \sum_y p(x, y) \quad (\text{discrete})$$

$$p(x) = \int p(x, y) dy \quad (\text{continuous})$$

4. **Product rule:** Rearranging the definition of conditional probability gives:

$$p(x, y) = p(x | y)p(y) = p(y | x)p(x)$$

Bayes theorem follows immediately by equating the two forms of the product rule:

$$p(y | x)p(x) = p(x | y)p(y)$$

$$p(y | x) = \frac{p(x | y)p(y)}{p(x)}$$

Interpretation: prior $p(y)$ is the belief about class y before seeing x ; likelihood $p(x | y)$ tells how compatible the observation is with class y ; posterior $p(y | x)$ is the updated belief after seeing x ; $p(x)$ is the normalization constant (evidence) that makes the posterior probabilities sum to one.

$$p(x) = \sum_y p(x | y)p(y)$$

Q6. Explain the Gaussian distribution and its importance in probability theory. For a dataset of i.i.d. observations, explain how Maximum Likelihood Estimation (MLE) can be used to estimate the mean and variance of a Gaussian distribution.

Answer. A Gaussian distribution is a continuous probability distribution characterized by a mean μ and variance σ^2 . It is important because it appears naturally in measurement noise, sampling theory, the central limit theorem, Bayesian modeling, and many machine-learning objectives.

$$\mathcal{N}(x \mid \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right)$$

Suppose $x_1, \dots, x_N$ are i.i.d. samples from $\mathcal{N}(\mu, \sigma^2)$. The likelihood is the product of the individual densities:

$$\mathcal{L}(\mu, \sigma^2) = \prod_{i=1}^N \mathcal{N}(x_i \mid \mu, \sigma^2)$$

It is easier to maximize the log-likelihood:

$$\ell(\mu, \sigma^2) = -\frac{N}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{i=1}^N (x_i - \mu)^2$$

Differentiate with respect to μ and set the derivative to zero:

$$\frac{\partial \ell}{\partial \mu} = \frac{1}{\sigma^2} \sum_{i=1}^N (x_i - \mu) = 0$$

$$\hat{\mu}_{\text{MLE}} = \frac{1}{N} \sum_{i=1}^N x_i$$

Differentiating with respect to σ^2 and setting the derivative to zero gives:

$$\hat{\sigma}_{\text{MLE}}^2 = \frac{1}{N} \sum_{i=1}^N (x_i - \hat{\mu}_{\text{MLE}})^2$$

The MLE variance uses $\frac{1}{N}$. The familiar unbiased sample variance uses $\frac{1}{N-1}$, which is a different estimator and should not be confused with the Gaussian MLE.

Q7. Explain the fundamental concepts of information theory. Define self-information, Shannon entropy, KL divergence, and cross-entropy. Discuss the properties of KL divergence and explain the relationship between cross-entropy and KL divergence.

Answer. Information theory quantifies uncertainty and the information contained in probabilistic events or distributions.

$$I(x) = -\log p(x)$$

A rare event has high self-information, while a likely event has low self-information.

$$H(P) = -\sum_x p(x) \log p(x)$$

Entropy measures the average uncertainty of a distribution. It is maximum when the distribution is uniform over a finite support.

$$D_{\text{KL}}(P\|Q) = \sum_x p(x) \log \frac{p(x)}{q(x)}$$

KL divergence measures how one distribution P differs from Q .

$$D_{\text{KL}}(P\|Q) \geq 0 \quad (\text{non-negativity / Gibbs inequality})$$

$$D_{\text{KL}}(P\|Q) = 0 \iff P = Q \text{ almost everywhere}$$

KL divergence is not symmetric in general: $D_{\text{KL}}(P\|Q) \neq D_{\text{KL}}(Q\|P)$. It is not a true distance metric because it does not satisfy symmetry and the triangle inequality.

$$H(P, Q) = -\sum_x p(x) \log q(x)$$

$$H(P, Q) = H(P) + D_{\text{KL}}(P\|Q)$$

Therefore, when P is fixed, minimizing cross-entropy with respect to Q is equivalent to minimizing KL divergence from P to Q because $H(P)$ is constant.

Q8. Medical diagnosis: Cancer C_1 has prior 0.10 and likelihood 0.80; No Cancer C_2 has prior 0.90 and likelihood 0.20. Using Bayes theorem, calculate posterior probabilities and classify the X-ray.

Answer. Let the observed X-ray result be x . The unnormalized joint probabilities are $p(x, C_1) = 0.10 \times 0.80 = 0.08$ and $p(x, C_2) = 0.90 \times 0.20 = 0.18$.

$$p(x) = 0.08 + 0.18 = 0.26$$

$$p(C_1 | x) = \frac{0.08}{0.26} \approx 0.3077 \approx 30.77\%$$

$$p(C_2 | x) = \frac{0.18}{0.26} \approx 0.6923 \approx 69.23\%$$

The posterior probability is therefore larger for C_2 (No Cancer). Under the usual maximum-posterior classification rule, the observation is classified as No Cancer.

Roles of the terms:

- **Prior** = initial prevalence/belief before observing the X-ray.
- **Likelihood** = compatibility of the observed X-ray result with each class.
- **Posterior** = updated probability after seeing the observation.
- **Normalization constant** = $p(x)=0.26$, which converts the unnormalized values 0.08 and 0.18 into probabilities that sum to 1.

Exam Tip

The important exam point is that a high likelihood for a class does not automatically make that class the final decision. The prior also matters.

PART II - DEEP GENERATIVE ARCHITECTURES

This section covers discriminative, generative and conditional generation, latent-manifold exploration, GAN/VAE/diffusion comparison, VAEs, the reparameterization trick, and a numerical latent-variable example.

Q1. Differentiate between discriminative $p_\theta(Y | X)$, generative $p_\theta(X)$, and conditioned generative $p_\theta(X | Y)$. Explain their objectives with examples. Also solve the classifier probability question.

Answer. Discriminative modeling focuses on predicting a label from an input. The model directly represents $p_\theta(Y | X)$. Generative modeling learns the data distribution $p_\theta(X)$, allowing new samples from the learned distribution. Conditioned generative modeling represents $p_\theta(X | Y)$, so the model generates an x conditioned on a class, text prompt, attribute, or other control variable.

Model	What is learned?	Typical objective	Example
Discriminative	$p_\theta(Y X)$	Maximize conditional likelihood / minimize classification loss	Image classifier
Generative	$p_\theta(X)$	Model or sample the data distribution	Unconditional image generator
Conditional generative	$p_\theta(X Y)$	Generate data consistent with conditioning variable	Class-conditional image generator

For the given classifier: Cat=0.10, Dog=0.75, Bird=0.15. The predicted class is Dog because it has the largest probability.

$$\mathcal{L}_{\text{NLL}} = -\log p(\text{actual class} | x) = -\log(0.75) \approx 0.28768 \approx 0.288$$

The NLL is low because the model assigns relatively high probability to the true class. If the true class had a tiny predicted probability, NLL would be much larger.

Q2. Discuss the importance of sampling, interpolation, and disentanglement in exploring the learned manifold of a deep generative model. Illustrate each concept with an example.

Answer. A generative model learns a mapping between latent variables and data space. Studying that mapping helps determine whether the model has learned meaningful structure.

- **Sampling:** draw z from the latent prior and compute $x = G(z)$. Good samples indicate that the learned generator can map ordinary latent points to realistic data. Example: sample several z vectors from $\mathcal{N}(0, I)$ and generate different faces.
- **Interpolation:** choose z_1 and z_2 and move continuously between them. If the resulting images change smoothly, the latent space contains meaningful continuous structure.

$$z(\alpha) = (1 - \alpha)z_1 + \alpha z_2, \quad 0 \leq \alpha \leq 1$$

- **Disentanglement:** different latent coordinates or latent directions ideally correspond to distinct semantic factors such as pose, hair color, or lighting. Example: changing one latent direction may alter head pose while leaving identity mostly unchanged. This makes the representation more interpretable and controllable.

Q3. Compare GANs, VAEs, and Denoising Diffusion Models in terms of architecture, training mechanism, sampling process, and ability to generate new data.

Aspect	GAN	VAE	Diffusion model
Architecture	Generator + discriminator	Encoder + decoder + latent variable	Forward noise process + learned reverse denoising network
Training	Adversarial two-player optimization	ELBO/reconstruction + KL regularization	Predict denoising/noise score across time steps
Sampling	Sample z and pass through G	Sample z from prior and decode	Start from noise and iteratively denoise
Strength	Sharp, realistic samples possible	Stable latent representation and probabilistic encoder	High sample quality and strong mode coverage
Typical drawback	Training instability / mode collapse	Blurrier samples in some settings	Sampling can require many denoising steps

All three are generative, but they learn generation in different ways: GANs learn through competition, VAEs through likelihood lower bounds and reconstruction, and diffusion models through a learned denoising process.

Q4. Explain the architecture and working of a Variational Autoencoder (VAE). Discuss how the encoder predicts the mean and covariance of a multivariate normal distribution, how the decoder reconstructs the input, and why the latent distribution is encouraged to follow $\mathcal{N}(0, I)$.

Answer. A VAE consists of an encoder $q_\phi(z | x)$ and a decoder $p_\theta(x | z)$. The encoder maps an input x to parameters of a probability distribution in latent space; the decoder maps a latent sample back into data space. **Encoder:** The encoder neural network outputs a mean vector $\mu(x)$ and either a covariance matrix $\Sigma(x)$ or, in the common diagonal-covariance implementation, log-variances. Thus:

$$q_\phi(z | x) = \mathcal{N}(z | \mu(x), \Sigma(x))$$

Latent sampling: a latent vector z is sampled from the encoder distribution. The decoder then produces $p_\theta(x | z)$ or a reconstruction x' . The reconstruction loss measures how well x' matches x . Why $\mathcal{N}(0, I)$? A standard normal prior gives the latent space a simple, smooth reference distribution. If the posterior stayed arbitrary for each training sample, random sampling at test time could land in unused or nonsensical regions. KL regularization keeps $q_\phi(z | x)$ close to $\mathcal{N}(0, I)$, making random generation from $\mathcal{N}(0, I)$ meaningful.

$$\mathcal{L}_{\text{VAE}} = \mathbb{E}_{q_\phi}[-\log p_\theta(x | z)] + D_{\text{KL}}(q_\phi(z | x) || p(z))$$

The first term encourages reconstruction quality. The second term regularizes the latent space and prevents each input from choosing a completely unrelated latent distribution.

Q5. Explain why direct sampling from $\mathcal{N}(\mu(X), \Sigma(X))$ creates a problem during backpropagation in a VAE. Explain the reparameterization trick and show how it enables gradient-based training.

Answer. A direct random sampling operation can interrupt the simple deterministic computational path used by backpropagation. The difficulty is that z is obtained by drawing a random value from a distribution whose parameters depend on x . The stochastic node itself is not conveniently differentiable with respect to μ and Σ in the naive implementation. The reparameterization trick separates the randomness from the learned parameters. For a diagonal Gaussian:

$$\epsilon \sim \mathcal{N}(0, I)$$

$$z = \mu(x) + \sigma(x)\epsilon$$

Now ϵ is an independent random variable and z is a deterministic differentiable function of μ , σ , and ϵ for a fixed sample of ϵ . Therefore gradients can flow through the operations:

$$\frac{\partial z}{\partial \mu} = 1, \quad \frac{\partial z}{\partial \sigma} = \epsilon$$

This allows standard backpropagation to update the encoder parameters while still performing stochastic latent sampling.

Q6. One-dimensional latent variable: $\mu=2$, $\sigma=0.5$, $\epsilon=-1.2$, with $z=\mu+\sigma*\epsilon$. **Decoder:** $x'=3z+1$. Calculate z and x' .

Answer. Substitute the given values.

$$z = 2 + (0.5)(-1.2) = 2 - 0.6 = 1.4$$

$$x' = 3(1.4) + 1 = 4.2 + 1 = 5.2$$

Therefore, the latent sample is $z=1.4$ and the generated output is $x'=5.2$.

PART III - MLE AND LATENT VARIABLE MODELS

The source asks why latent variables are useful, how unobserved factors can represent high-level variation, and how Jensen inequality leads to the ELBO and its relationship with KL divergence. It also gives a four-configuration posterior calculation.

Q1. Explain the motivation for using latent variable models. For the image example, answer (a)-(d).

Answer. Latent variable models introduce hidden variables z that explain underlying structure in observed data x . This is useful because high-dimensional observations often arise from a smaller number of meaningful causes or factors. Instead of modeling every pixel directly, a model can first represent latent factors such as identity, pose, lighting, gender, or hair color and then generate the image from them. **(a) Observed versus latent variables:** x is the observed image and z contains hidden variables or latent factors. The training data may contain x but not direct labels for z . **(b) High-level factors:** z can encode semantically meaningful variations. For example, one component or direction of z may correlate with pose, another with hair color, and another with lighting. In an ideal disentangled representation, changing one factor would have a focused semantic effect. **(c) Generative model $p(x | z)$:** Given a latent state z , the decoder models how an image could be generated. This conditional problem can be simpler because z summarizes the important causes. The full data distribution is obtained by integrating/summing over z :

$$p(x) = \int p(x | z)p(z) dz$$

(d) Posterior $p(z | x)$: The posterior tells us which latent configurations explain an observed image. Learning or approximating this posterior provides an unsupervised representation of the data. The latent vector can then be used for clustering, visualization, interpolation, reconstruction, or downstream tasks.

Q2. For a latent variable model, explain why Jensen inequality applies, introduce $q(z)$, derive the ELBO, and explain why it is a lower bound.

Answer. The marginal likelihood contains a sum/integral over the unobserved latent variable. This makes direct optimization difficult. We therefore introduce any distribution $q(z)$ whose support covers the relevant posterior region and rewrite the likelihood using $q(z)$.

$$\begin{aligned}\log p(x) &= \log \int p(x, z) dz \\ &= \log \int q(z) \frac{p(x, z)}{q(z)} dz\end{aligned}$$

This is the log of an expectation under $q(z)$:

$$\log p(x) = \log \mathbb{E}_q \left[\frac{p(x, z)}{q(z)} \right]$$

The logarithm is concave, so Jensen inequality gives $\log \mathbb{E}[Y] \geq \mathbb{E}[\log Y]$. Therefore:

$$\log p(x) \geq \mathbb{E}_q[\log p(x, z) - \log q(z)]$$

The right-hand side is the Evidence Lower Bound:

$$\text{ELBO} = \mathbb{E}_q[\log p(x, z) - \log q(z)]$$

Using $p(x, z) = p(x | z)p(z)$, an equivalent form is:

$$\text{ELBO} = \mathbb{E}_q[\log p(x | z)] - D_{\text{KL}}(q(z) \| p(z))$$

It is called a lower bound because the KL divergence is non-negative, which implies the ELBO cannot exceed the true log-likelihood.

Q3. For the relationship $\log p(x) = \text{ELBO} + D_{\text{KL}}(q(z | x) || p(z | x))$, explain non-negativity, derive the relationship, state the equality condition, and explain what happens as $q_\phi(z | x)$ improves.

Answer. Start from the KL divergence between the variational distribution and the true posterior.

$$D_{\text{KL}}(q(z | x) || p(z | x)) = \mathbb{E}_q \left[\log \frac{q(z | x)}{p(z | x)} \right]$$

Because $p(z | x) = \frac{p(x, z)}{p(x)}$:

$$D_{\text{KL}} = \mathbb{E}_q [\log q(z | x) - \log p(x, z) + \log p(x)]$$

$$= \log p(x) - \mathbb{E}_q [\log p(x, z) - \log q(z | x)]$$

$$= \log p(x) - \text{ELBO}$$

Hence:

$$\log p(x) = \text{ELBO} + D_{\text{KL}}(q(z | x) || p(z | x))$$

- (a) Since $D_{\text{KL}} \geq 0$, $\log p(x) \geq \text{ELBO}$. Hence the ELBO is a lower bound.
- (b) The derivation follows from substituting Bayes rule into the KL expression as shown above.
- (c) Equality occurs when $D_{\text{KL}}(q(z | x) || p(z | x)) = 0$, which means $q(z | x) = p(z | x)$ almost everywhere.
- (d) As q_ϕ becomes a better approximation to the true posterior, the KL divergence decreases toward zero and the ELBO increases toward the true log-likelihood.

Q4. A latent variable z has four possible configurations with joint probabilities $p(x, z_1)=0.10$, $p(x, z_2)=0.25$, $p(x, z_3)=0.05$, $p(x, z_4)=0.20$. Calculate the marginal, posterior probabilities, the most probable configuration, and interpret the result.

Answer. First add the joint probabilities to obtain the marginal likelihood.

$$p(x) = 0.10 + 0.25 + 0.05 + 0.20 = 0.60$$

Now divide each joint probability by 0.60:

$$p(z_1 | x) = \frac{0.10}{0.60} = 0.1667$$

$$p(z_2 | x) = \frac{0.25}{0.60} = 0.4167$$

$$p(z_3 | x) = \frac{0.05}{0.60} = 0.0833$$

$$p(z_4 | x) = \frac{0.20}{0.60} = 0.3333$$

The posterior sums to approximately 1.0000. The most probable configuration is z_2 with posterior probability about 41.67%. In clustering terms, z_2 is the latent cluster/state that best explains x among the four candidates. In representation learning, it indicates that the observed sample is most strongly associated with the features encoded by z_2 , while the other configurations remain alternative explanations with smaller posterior mass.

PART IV - GENERATIVE ADVERSARIAL NETWORKS

The GAN section covers architecture, discriminator and generator losses, numerical calculations, practical training problems, the optimal-discriminator/JSD derivation, and evaluation through Inception Score and FID.

Q1. Explain the architecture and working principle of a GAN. Describe G and D, the minimax game, and the objective function.

Answer. A Generative Adversarial Network consists of two neural networks: a generator G_θ and a discriminator D_ϕ . The generator maps a random latent vector z into a synthetic sample $G(z)$. The discriminator receives either a real sample x from the data distribution or a generated sample and outputs a probability that the input is real. Training alternates between two goals. The discriminator learns to assign high probability to real data and low probability to generated data. The generator learns to produce samples that the discriminator classifies as real.

$$V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_{z \sim p(z)} [\log(1 - D(G(z)))]$$

The minimax game is $\min_G \max_D V(D, G)$:

- **First term:** rewards D for correctly recognizing real samples.
- **Second term:** rewards D for correctly rejecting generated samples.

D maximizes the objective because it wants accurate discrimination. G minimizes it in the original minimax formulation because it wants the discriminator to fail on generated samples. In practical implementations, the non-saturating generator loss $-\mathbb{E}_z [\log D(G(z))]$ is often used because it gives stronger gradients early in training.

Q2. Discriminator outputs $D(x)=0.9$ for a real sample and $D(G(z))=0.2$ for a fake sample. Calculate $L_D=-[\log D(x)+\log(1-D(G(z)))]$ and interpret it.

Answer. Substitute the values:

$$\begin{aligned}\mathcal{L}_D &= -[\log(0.9) + \log(1 - 0.2)] \\ &= -[\log(0.9) + \log(0.8)] = -\log(0.72)\end{aligned}$$

L_D approx 0.3285 **Interpretation:** both outputs are on the correct side of the decision boundary: the real sample is assigned high real probability and the fake sample is assigned low real probability. The relatively small positive BCE loss therefore indicates that the discriminator is performing well on these two examples.

Q3. For x_1, x_2 real with $D=0.9, 0.8$ and two fake samples with $D=0.2, 0.1$, calculate the discriminator objective $V(G, D)$ using natural logarithms. Determine whether D is performing well.

Answer. Use the minibatch form of the GAN value function:

$$\begin{aligned} V &= \frac{1}{2}[\log(0.9) + \log(0.8)] + \frac{1}{2}[\log(1 - 0.2) + \log(1 - 0.1)] \\ &= 1/2[\log(0.9) + \log(0.8)] + 1/2[\log(0.8) + \log(0.9)] \\ &= \log(0.72) \approx -0.3285 \end{aligned}$$

The discriminator is performing well on this minibatch because it gives high real probabilities to the real samples and low real probabilities to the fake samples. The value is negative because logarithms of numbers between 0 and 1 are negative.

Q4. Generator outputs $D(G(z))=[0.1,0.3,0.7,0.5]$. Using $L_G=-(1/4) \sum \log D(G(z_i))$, calculate the generator loss, interpret it, then recalculate for $[0.6,0.7,0.8,0.9]$.

Answer. Case 1:

$$L_G = -\frac{1}{4}[\log(0.1) + \log(0.3) + \log(0.7) + \log(0.5)]$$

$$= -1/4 \log(0.1 \cdot 0.3 \cdot 0.7 \cdot 0.5) = -1/4 \log(0.0105) \approx 1.1391 \text{ Case 2:}$$

$$L_G = -\frac{1}{4}[\log(0.6) + \log(0.7) + \log(0.8) + \log(0.9)]$$

$= -1/4 \log(0.3024) \approx 0.2990$ A lower non-saturating generator loss means the discriminator is assigning higher real probabilities to generated samples, so the generator is currently doing better at fooling D. Thus the second case is associated with a substantially smaller generator loss than the first case. This is an interpretation of the given loss, not a claim that one minibatch proves overall model quality.

Q5. Discuss major practical GAN training challenges, including unstable optimization and mode collapse. Explain oscillating losses and empirical mitigation approaches.

Answer. GAN training is a two-player game rather than an ordinary single-objective optimization problem. The generator and discriminator continually adapt to one another, so the target seen by one network changes as the other is updated.

- **Unstable optimization:** If D becomes too strong, G may receive weak or unhelpful gradients in some formulations. If G improves too quickly, D can become inaccurate. The balance can therefore oscillate rather than converge smoothly.
- **Oscillating losses:** GAN losses need not decrease monotonically. A decrease in discriminator loss can make generator loss increase and vice versa. Moreover, the numerical value of a GAN loss depends on the current opponent, so it is not a standalone quality score.
- **Mode collapse:** The generator maps many different latent vectors to similar outputs. The samples may look realistic but fail to cover the diversity of the real data distribution. Example: a face generator repeatedly produces faces with very similar pose, hairstyle, and background while ignoring other valid variations.
- **Empirical mitigation:** commonly used approaches include improved discriminator regularization, careful learning-rate/ β tuning, minibatch-based diversity mechanisms, feature matching, alternative adversarial objectives, spectral normalization, gradient penalties, data augmentation, and architectural changes. No single method guarantees complete elimination of collapse.

Q6. Consider a GAN with an optimal discriminator D^* . Show that the value function can be expressed using Jensen-Shannon Divergence (JSD). Define JSD, state the optimum p_G , and the corresponding objective value.

Answer. For a fixed generator distribution p_G , the optimal discriminator is obtained pointwise by maximizing $p_{\text{data}}(x) \log D(x) + p_G(x) \log(1 - D(x))$. Differentiating with respect to $D(x)$ and setting the derivative to zero gives:

$$D_G^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_G(x)}$$

Substituting D_G^* into the GAN value function gives:

$$V(D_G^*, G) = \int p_{\text{data}} \log \frac{p_{\text{data}}}{p_{\text{data}} + p_G} dx + \int p_G \log \frac{p_G}{p_{\text{data}} + p_G} dx$$

$$\text{Let } M = (p_{\text{data}} + p_G)/2. \text{ Since } p_{\text{data}} + p_G = 2M :$$

$$V(D_G^*, G) = -\log 4 + 2 \text{ JSD}(p_{\text{data}} \| p_G)$$

where

$$\text{JSD}(P \| Q) = \frac{1}{2} D_{\text{KL}}(P \| M) + \frac{1}{2} D_{\text{KL}}(Q \| M), \quad M = \frac{P + Q}{2}$$

JSD is symmetric, unlike ordinary KL divergence. JSD is non-negative. JSD is zero exactly when $P=Q$ (under the usual distributional assumptions). With natural logarithms, JSD is bounded above by $\log 2$. Therefore the global minimum over G occurs when $p_G = p_{\text{data}}$, because then $\text{JSD} = 0$. The corresponding value of the original GAN objective is:

$$V(D_G^*, G) = -\log 4$$

Q7. Explain why evaluating generative models is challenging and how evaluation depends on the intended task. Cover density estimation, sampling, latent representation learning, and downstream applications.

Answer. No single evaluation score captures all aspects of a generative model. A model may have good likelihood but produce visually weak samples, or produce visually impressive samples while missing modes of the true distribution. Evaluation must therefore match the intended use.

- **Density estimation:** Evaluate likelihood or held-out log-likelihood when the model explicitly provides a tractable or comparable density. The goal is whether the learned probability model assigns appropriate probability to unseen data.
- **Sampling/generation:** Evaluate realism, diversity, fidelity, coverage, and human or task-specific quality. Image metrics such as IS and FID are common, but they are imperfect proxies.
- **Latent representation learning:** Evaluate whether latent codes capture meaningful factors, support smooth interpolation, clustering, reconstruction, or disentanglement.
- **Downstream applications:** Evaluate the actual task, such as semi-supervised classification, image translation, retrieval, completion, augmentation, or domain adaptation. The generated data or latent representation should improve the target application rather than only score well on a generic metric.

Q8. Explain the Inception Score (IS). Discuss high sharpness and high diversity using $c(y|x)$ and $c(y)$.

Answer. The Inception Score uses a pretrained classifier to measure two desirable properties. For each generated sample x , $c(y|x)$ is the classifier prediction over labels. A high-quality image should produce a confident prediction, so $c(y|x)$ has low entropy. Across many generated images, the marginal $c(y)=E_x[c(y|x)]$ should have high entropy so that the generator covers many classes.

$$IS = \exp(\mathbb{E}_x [D_{KL}(c(y | x) \| c(y))])$$

Sharpness/fidelity is associated with low uncertainty in $c(y|x)$. Diversity is associated with a broad marginal $c(y)$. The KL divergence is high when each individual prediction is sharp while the collection covers varied classes, producing a higher Inception Score. The score is not a direct measurement of pixel-level realism and depends on the chosen classifier.

Q9. Explain FID, write the formula, explain its terms, interpret a low FID, and compare it conceptually with IS.

Answer. Frechet Inception Distance compares feature distributions of real and generated samples. The feature vectors are commonly obtained from an intermediate layer of a pretrained image classifier and approximated by multivariate Gaussians.

$$\text{FID} = \|\mu_G - \mu_T\|_2^2 + \text{Tr}\left(\Sigma_G + \Sigma_T - 2(\Sigma_G \Sigma_T)^{1/2}\right)$$

- μ_G, Σ_G : mean and covariance of generated-sample features.
- μ_T, Σ_T : mean and covariance of test/real-sample features.
- **First term:** squared difference between feature means.
- **Trace term:** difference between feature covariances.

A lower FID means the generated feature distribution is closer to the real/test feature distribution under this Gaussian feature-space approximation. FID compares generated and real distributions directly, whereas IS uses only generated samples and asks whether they are individually classifiable and collectively diverse according to a classifier.

PART V - VARIANTS OF GAN

The uploaded source then covers cGAN, BiGAN, CoGAN, CycleGAN, latent interpolation/steerability, and bidirectional latent inference.

Q1. Explain the architecture and working principle of a Conditional GAN (cGAN). How does conditioning on class information modify a conventional GAN? Explain text-to-image use.

Answer. A conditional GAN adds an observed condition y , such as a class label or text description, to both the generator and discriminator. Instead of learning $G(z)$, the generator learns $G(z, y)$, while the discriminator evaluates whether a sample is real and whether it is consistent with y .

$$V(D, G) = \mathbb{E}_{x, y \sim p_{\text{data}}} [\log D(x, y)] + \mathbb{E}_{z \sim p(z), y \sim p(y)} [\log(1 - D(G(z, y), y))]$$

For class-conditional image generation, y could be the label “cat” or “car”. The generator receives z and y and creates an image matching the requested class. For text-to-image synthesis, y is replaced by a learned text embedding; the generator uses that embedding as semantic conditioning and D checks image-text compatibility as part of its discrimination task.

Q2. Explain the motivation behind BiGAN. Describe how G and E learn the joint distributions of $(\mathbf{X}, \mathbf{Z})$, and state the inverse-mapping condition.

Answer. A conventional GAN learns to map z to x but does not naturally provide the inverse mapping from an observed x back to a latent z . A Bidirectional GAN (BiGAN) introduces an encoder $E(x)$ to infer a latent representation from real data. The discriminator receives pairs rather than single samples. From the generator path it sees $(z, G(z))$. From the encoder path it sees $(E(x), x)$. The discriminator learns to distinguish these two joint distributions, while G and E learn together to make them indistinguishable.

- **Joint 1 (generator path):** $(z, G(z))$
- **Joint 2 (encoder path):** $(E(x), x)$

When training is successful and the mappings are compatible, E and G become approximately inverse on the data manifold: $E(G(z)) \approx z$ and $G(E(x)) \approx x$ for relevant samples. BiGAN therefore supports both sample generation and latent inference.

Q3. CoGAN for two semantically related but unpaired domains X_A and X_B : explain learning without correspondence supervision, weight sharing, and one limitation.

Answer. CoGAN uses two GAN-style networks, one per domain, and couples them through shared parameters. The domains may contain related semantic objects but there are no paired examples saying which image in A corresponds to which image in B . The domain-specific parts capture low-level appearance differences, while shared layers encourage the two networks to encode common high-level semantics. Because shared parameters are trained from both domains, the model can learn a common latent structure without explicit image-to-image correspondences. **Weight sharing is an inductive bias:** if high-level concepts such as “object category” are common across domains, sharing deeper layers encourages the networks to discover representations useful to both. Earlier layers are often left separate because low-level statistics such as color, texture, and contrast may differ by domain. **Limitation:** CoGAN relies on the assumption that the two domains share useful high-level semantics. If their underlying structures differ strongly, the shared representation may be inappropriate; training can also be sensitive to the domain gap and adversarial optimization.

Q4. Explain CycleGAN for unpaired image-to-image translation. Define $G_{A \rightarrow B}$ and $G_{B \rightarrow A}$, adversarial loss, cycle consistency, identity loss, and a limitation of cycle loss alone.

Answer. CycleGAN addresses image-to-image translation when paired examples are unavailable. It contains two generators and two discriminators: $G : X \rightarrow Y$, $F : Y \rightarrow X$, D_Y distinguishes real Y from $G(X)$, and D_X distinguishes real X from $F(Y)$. Adversarial loss in the $X \rightarrow Y$ direction encourages $G(x)$ to look like a real Y image. A second adversarial loss does the analogous job for $F(y)$ in the $Y \rightarrow X$ direction.

$$\mathcal{L}_{\text{adv}}(G, D_Y, X, Y) = \mathbb{E}_y[\log D_Y(y)] + \mathbb{E}_x[\log(1 - D_Y(G(x)))]$$

Cycle consistency requires that translating to the other domain and back approximately reproduces the input: $F(G(x)) \approx x$, $G(F(y)) \approx y$

$$\mathcal{L}_{\text{cyc}} = \mathbb{E}_x[\|F(G(x)) - x\|_1] + \mathbb{E}_y[\|G(F(y)) - y\|_1]$$

Identity loss can be added when preserving already-correct images is desirable, for example when an image from Y is passed through G and should remain close to itself. Cycle consistency alone does not uniquely determine the intended semantic mapping: multiple mappings can satisfy the cycle constraint, including undesirable ones. Adversarial, identity, and architectural biases therefore work together.

Q5. Explain linear interpolation in GAN latent space and steerability. Given z_1, z_2 and α in $[0,1]$, derive the interpolated vector and explain transformations such as zooming, shifting, brightness, and rotation.

Answer. For two latent vectors z_1 and z_2 , the simplest interpolation is linear interpolation:

$$z(\alpha) = (1 - \alpha)z_1 + \alpha z_2, 0 \leq \alpha \leq 1$$

At $\alpha=0$, $z=z_1$. At $\alpha=1$, $z=z_2$. Intermediate values generate intermediate images through $G(z(\alpha))$. If the latent manifold is smooth, the visual attributes also change smoothly. Steerability means that meaningful image transformations can be represented approximately by moving along learned directions in latent space. Suppose a direction d_{zoom} changes apparent scale. Then $z_{\text{prime}}=z+t*d_{\text{zoom}}$ can increase or decrease zoom. Similarly, separate directions may correspond approximately to horizontal shift, brightness, rotation, or semantic attributes. In a well-structured latent space, these directions can be combined to control several factors. In practice, the correspondence is approximate and depends on the model and training data.

Q6. Explain BiGAN motivation, compare with conventional GAN, roles of G/E/D, joint discrimination, and how a trained BiGAN supports generation and inference.

Answer. The key addition in BiGAN is an encoder E that maps observed data to latent space. A conventional GAN has $G:z \rightarrow x$ but no explicit learned inverse $x \rightarrow z$. BiGAN learns both directions through a joint adversarial game. G : generates x from z . E : maps x to a latent representation $\hat{z}$. D : distinguishes joint pairs $(z, G(z))$ from $(E(x), x)$. The discriminator never needs to decide whether z or x is individually real. Instead, it checks whether the pair has been produced by the generator side or the encoder/data side. When the joint distributions match, the encoder and generator are approximately inverse on the relevant manifolds. After training, generation is performed by sampling z and computing $G(z)$. Representation inference is performed by taking a real x and computing $E(x)$. This makes BiGAN useful when both synthesis and latent embeddings are required.

Q7. Explain the problem of image-to-image translation $X \leftrightarrow Y$ without paired examples. Describe CycleGAN architecture and adversarial training in both directions.

Answer. Without paired examples, there is no direct supervised loss of the form $\|G(x_i) - y_i\|$ because the correct y_i paired with x_i is unknown. CycleGAN addresses this by learning two mappings: $G : X \rightarrow Y$ and $F : Y \rightarrow X$, with discriminators D_Y and D_X . For $G : X \rightarrow Y$, D_Y sees real y samples and generated $G(x)$ samples. The adversarial objective encourages $G(x)$ to match the distribution of Y . For $F : Y \rightarrow X$, D_X sees real x samples and generated $F(y)$ samples. The adversarial objective encourages $F(y)$ to match the distribution of X . The cycle losses link the two directions so that the learned mapping preserves information relevant to the original sample rather than producing arbitrary samples from the target domain.

Q8. Explain cycle consistency and derive the complete CycleGAN objective. Explain the role of λ and both adversarial losses.

Answer. Cycle consistency is the constraint that translating from one domain to the other and then back should approximately reconstruct the original input. It is central because unpaired data do not supply a direct ground-truth target.

$$\mathcal{L}_{\text{GAN}}(G, D_Y, X, Y) = \mathbb{E}_y[\log D_Y(y)] + \mathbb{E}_x[\log(1 - D_Y(G(x)))]$$

$$\mathcal{L}_{\text{GAN}}(F, D_X, Y, X) = \mathbb{E}_x[\log D_X(x)] + \mathbb{E}_y[\log(1 - D_X(F(y)))]$$

$$\mathcal{L}_{\text{cyc}}(G, F) = \mathbb{E}_x[\|F(G(x)) - x\|_1] + \mathbb{E}_y[\|G(F(y)) - y\|_1]$$

$$\mathcal{L}_{\text{CycleGAN}} = \mathcal{L}_{\text{GAN}}(G, D_Y, X, Y) + \mathcal{L}_{\text{GAN}}(F, D_X, Y, X) + \lambda \mathcal{L}_{\text{cyc}}(G, F)$$

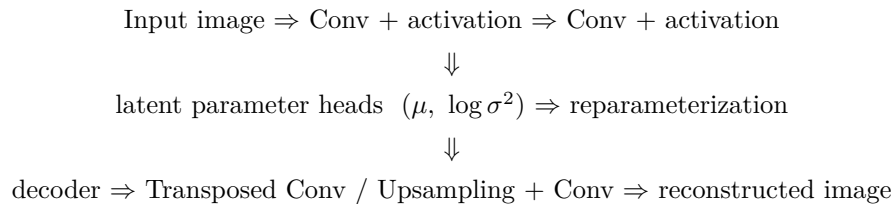
The two adversarial terms make translated samples look realistic in their respective target domains. The cycle term prevents the mappings from ignoring the input structure. Lambda controls the trade-off: a larger λ places more emphasis on reconstruction through the cycle, while a smaller λ gives relatively greater weight to adversarial realism. Some implementations add an identity term as a further regularizer, but the complete objective requested in the source explicitly centers on the two adversarial losses plus the λ -weighted cycle loss.

PART VI - VARIANTS OF VAE

The final section of the source covers convolutional VAEs, CVAEs, β -VAE, Ladder VAE, a β -VAE numerical, and a parameter-count comparison between fully connected and convolutional encoders.

Q1. How does a Convolutional VAE address the curse of dimensionality? Illustrate the architecture.

Answer. A fully connected layer applied directly to a high-resolution image treats every input pixel as an independent input to every output neuron. This creates a huge parameter count and ignores local spatial structure. A convolutional VAE replaces the early fully connected processing with convolutional layers that use local receptive fields and shared weights. Typical architecture:



- **Local connectivity:** each filter sees a small neighborhood.
- **Weight sharing:** the same filter is reused across the image, sharply reducing parameters.
- **Hierarchical features:** early layers learn edges/ textures; deeper layers learn larger shapes and semantic structures.

Spatial structure is preserved through feature maps rather than immediately flattening the image. Thus, the convolutional encoder makes high-dimensional image modeling more parameter-efficient and statistically appropriate. The decoder reverses this hierarchy to reconstruct the image.

Q2. Explain the architecture and objective of a Conditional VAE (CVAE). How does conditioning influence the latent representation and generated output?

Answer. A CVAE introduces an observed conditioning variable y such as a class label, attribute, or text embedding. The encoder models $q_\phi(z | x, y)$, the decoder models $p_\theta(x | z, y)$, and the prior can also be conditioned when appropriate.

$$\mathcal{L}_{\text{CVAE}} = \mathbb{E}_{q_\phi}[-\log p_\theta(x | z, y)] + D_{\text{KL}}(q_\phi(z | x, y) \| p(z | y))$$

The condition tells the encoder what context must be explained and tells the decoder what kind of sample to generate. For example, with y ="digit 7", the decoder can generate an image that resembles a 7 while z captures the remaining variation such as stroke width or style. Conditioning therefore makes generation controllable rather than purely random with respect to the target attribute.

Q3. Explain β -VAE, write the modified objective, explain β , and discuss disentanglement.

Answer. Beta-VAE modifies the standard VAE by multiplying the KL divergence term by a hyperparameter β . The standard VAE balances reconstruction and latent regularization with weight 1. Beta-VAE changes this trade-off.

$$\mathcal{L}_{\beta\text{-VAE}} = \mathbb{E}_{q_\phi}[-\log p_\theta(x | z)] + \beta D_{\text{KL}}(q_\phi(z | x) \| p(z))$$

When $\beta = 1$, this is the standard VAE objective. For $\beta > 1$, the KL term is emphasized more strongly. This limits the amount of information that can pass through the latent code and encourages the latent variables to conform more closely to the factorized prior $\mathcal{N}(0, I)$. Because the prior factorizes and the KL pressure discourages arbitrary correlations and excessive information, β -VAE often encourages disentangled factors. However, stronger regularization can increase reconstruction error or remove fine-grained information. Therefore β controls an information-capacity trade-off rather than guaranteeing perfect disentanglement.

Q4. Explain the motivation for Ladder VAE (LVAE). How does it learn a hierarchical latent representation? Compare conventional hierarchical VAE and ladder architecture.

Answer. A Ladder VAE is motivated by the observation that real data often have hierarchical structure: low-level details such as edges may depend on medium-level shapes, which in turn depend on high-level concepts. A single latent layer may not represent this hierarchy efficiently. A hierarchical VAE may contain multiple latent variables $z_1, z_2, \dots$ at different abstraction levels. In a conventional arrangement, the encoder processes information upward while the decoder samples latents downward. The ladder architecture adds top-down and bottom-up pathways so that information at corresponding levels can be combined, allowing each latent variable to be refined using both inference evidence and generative context. **Conceptual flow:** $x \rightarrow$ bottom-up features $\rightarrow z_1 \rightarrow z_2 \rightarrow z_3$ (high-level) **Top-down path:** $z_3 \rightarrow z_2 \rightarrow z_1 \rightarrow$ reconstruction **The advantage is hierarchical representation:** higher latents can capture broad semantic factors, while lower latents preserve local detail. Ladder structures also help inference networks combine bottom-up evidence with top-down priors, improving the modeling of hierarchical uncertainty.

Q5. β -VAE reconstruction loss = 18.5 and KL loss = 2.4. Objective = $L_{\text{reconstruction}} + \beta L_{\text{KL}}$. Calculate for $\beta=1,2,4$ and explain the effect of increasing β .

Answer. Use $L = 18.5 + \beta(2.4)$.

$$\beta = 1 : L = 18.5 + 2.4 = 20.9$$

$$\beta = 2 : L = 18.5 + 4.8 = 23.3$$

$$\beta = 4 : L = 18.5 + 9.6 = 28.1$$

As β increases, the KL divergence receives more weight relative to reconstruction. The model is pushed harder toward the prior $\mathcal{N}(0, I)$, which reduces the effective information capacity of the latent channel and can encourage a more structured/disentangled representation. The trade-off is that reconstruction can become less accurate because less information is allowed to be encoded in z .

Q6. A vanilla VAE uses a fully connected layer from 64x64x3 to 512. A convolutional VAE first reduces the image to 8x8x64 and then uses the same 512-dimensional fully connected layer. Calculate weights and the reduction.

Answer. First, flatten the image.

$$\text{Input size} = 64 \times 64 \times 3 = 12,288$$

$$W_{\text{vanilla}} = 12,288 \times 512 = 6,291,456$$

For the convolutional representation:

$$\text{Feature size} = 8 \times 8 \times 64 = 4,096$$

$$W_{\text{conv}} = 4,096 \times 512 = 2,097,152$$

Reduction:

$$\text{Reduction} = 6,291,456 - 2,097,152 = 4,194,304 \text{ weights}$$

$$\text{Percentage reduction} = \frac{4,194,304}{6,291,456} \times 100 \approx 66.67\%$$

Thus the convolutional approach uses about one-third as many weights in this fully connected stage. The main reason convolutional architectures are preferable for images is even broader: convolutions exploit locality and weight sharing before the final dense layer, so they can reduce both parameter count and unnecessary dependence on exact pixel positions.

FINAL RAPID REVISION

This section condenses the formulas and numerical results that are most useful during last-minute revision. The full derivations remain in the preceding sections.

1. Probability and Bayes

$$p(x, y) = p(x | y)p(y) = p(y | x)p(x)$$

$$p(y | x) = \frac{p(x | y)p(y)}{p(x)}$$

$$p(x) = \sum_y p(x | y)p(y)$$

Medical Bayes numerical:

$$p(C_1 | x) = 0.3077, \quad p(C_2 | x) = 0.6923$$

Decision under MAP: No Cancer.

2. Gaussian MLE

$$\hat{\mu} = \frac{1}{N} \sum_{i=1}^N x_i$$

$$\hat{\sigma}^2 = \frac{1}{N} \sum_{i=1}^N (x_i - \hat{\mu})^2$$

3. Information theory

$$I(x) = -\log p(x)$$

$$H(P) = -\sum_x p(x) \log p(x)$$

$$D_{\text{KL}}(P \| Q) = \sum_x p(x) \log \frac{p(x)}{q(x)}$$

$$H(P, Q) = H(P) + D_{\text{KL}}(P \| Q)$$

4. VAE

$$\mathcal{L}_{\text{VAE}} = \mathbb{E}_q [-\log p(x | z)] + D_{\text{KL}}(q(z | x) \| p(z))$$

$$z = \mu + \sigma\epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

Reparameterization numerical: $z = 1.4$, $x' = 5.2$.

5. GAN

$$V(D, G) = \mathbb{E}_x [\log D(x)] + \mathbb{E}_z [\log (1 - D(G(z)))]$$

$$D^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_G(x)}$$

$$V(D^*, G) = -\log 4 + 2 \text{JSD}(p_{\text{data}} \| p_G)$$

At optimum: $p_G = p_{\text{data}}$ and $V = -\log 4$.

GAN numericals: discriminator BCE = 0.3285; minibatch $V = -0.3285$; generator losses = 1.1391 and 0.2990.

6. GAN evaluation

$$\text{IS} = \exp(\mathbb{E}_x [D_{\text{KL}}(c(y | x) \| c(y))])$$

$$\text{FID} = \|\mu_G - \mu_T\|_2^2 + \text{Tr}(\Sigma_G + \Sigma_T - 2(\Sigma_G \Sigma_T)^{1/2})$$

7. Latent models

$$\text{ELBO} = \mathbb{E}_q [\log p(x, z) - \log q(z | x)]$$

$$\log p(x) = \text{ELBO} + D_{\text{KL}}(q(z | x) \| p(z | x))$$

Latent posterior numerical:

$$p(x) = 0.60$$

$$[p(z_1 | x), p(z_2 | x), p(z_3 | x), p(z_4 | x)] = [0.1667, 0.4167, 0.0833, 0.3333]$$

Most probable configuration: z_2 .

8. CycleGAN

$$\mathcal{L} = \mathcal{L}_{\text{GAN}}(G, D_Y) + \mathcal{L}_{\text{GAN}}(F, D_X) + \lambda \mathcal{L}_{\text{cyc}}$$

$$\mathcal{L}_{\text{cyc}} = \mathbb{E}_x [\|F(G(x)) - x\|_1] + \mathbb{E}_y [\|G(F(y)) - y\|_1]$$

9. β -VAE numerical

$$\beta = 1 \Rightarrow 20.9, \quad \beta = 2 \Rightarrow 23.3, \quad \beta = 4 \Rightarrow 28.1$$

10. Parameter-count numerical

$$64 \times 64 \times 3 \rightarrow 512 : \quad 6,291,456 \text{ weights}$$

$$8 \times 8 \times 64 \rightarrow 512 : \quad 2,097,152 \text{ weights}$$

$$\text{Reduction} = 4,194,304 \text{ weights} \approx 66.67\%$$

Exam Tip

For long-answer questions, use the pattern: definition $\rightarrow$ architecture/equation $\rightarrow$ working/derivation $\rightarrow$ interpretation $\rightarrow$ one example or limitation. For numerical questions, write the formula first, substitute the values, compute, and then state the final interpretation.